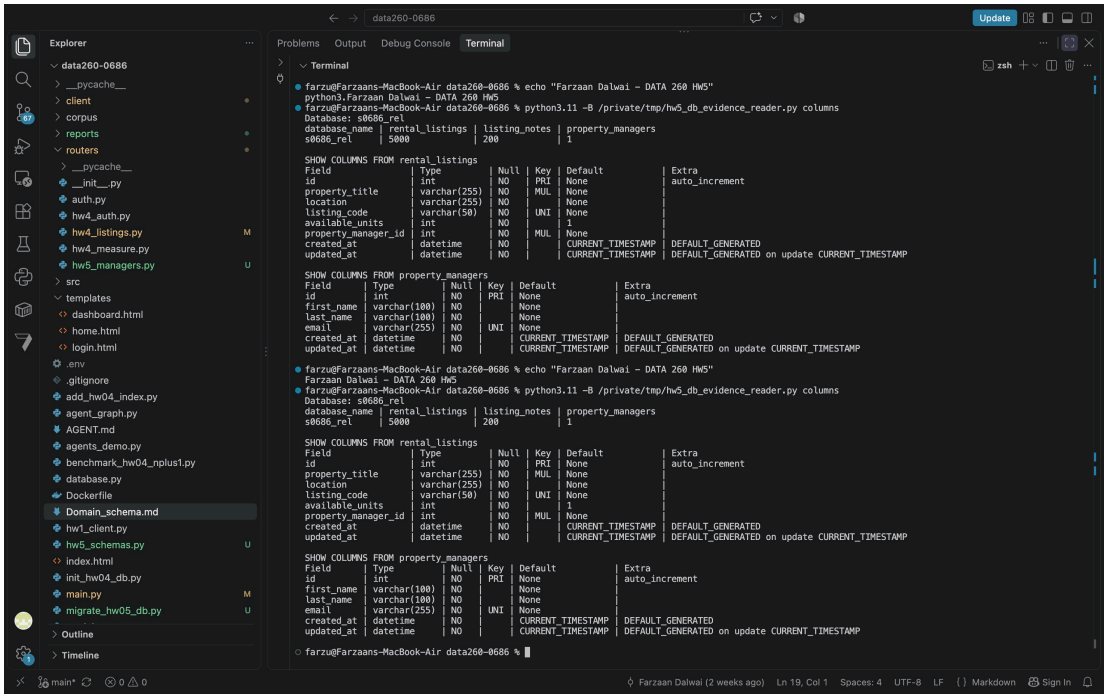
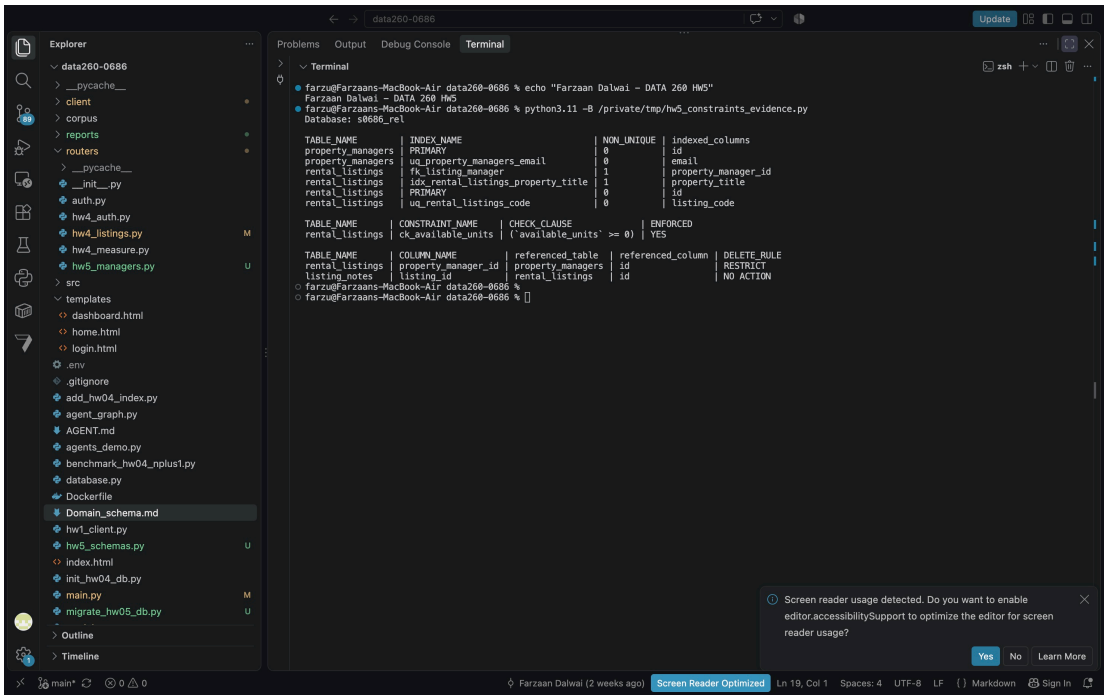
Farzaan Dalwai
DATA 260 - Homework 5
GitHub: https://github.com/farzaandalwai/data260-0686
SID4 = 0686
PORT_BASE = 8686
PREFIX = s0686
SEED = 686
VERIFY_SEED = 260686
DOMAIN_ID = 6
Domain = Rental Housing Listings
Hardware: MacBook Air, Apple M4, 10 cores, 24 GB memory
Local model: llama3.2:latest
Tagged commit: PENDING

Part 1

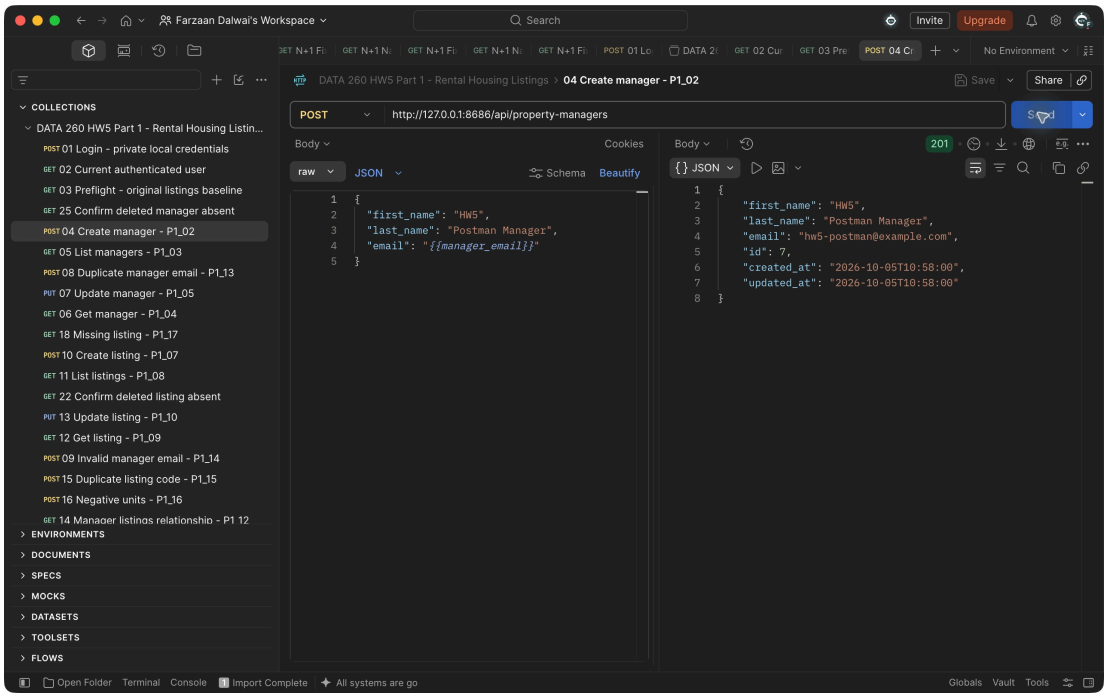
Database schema



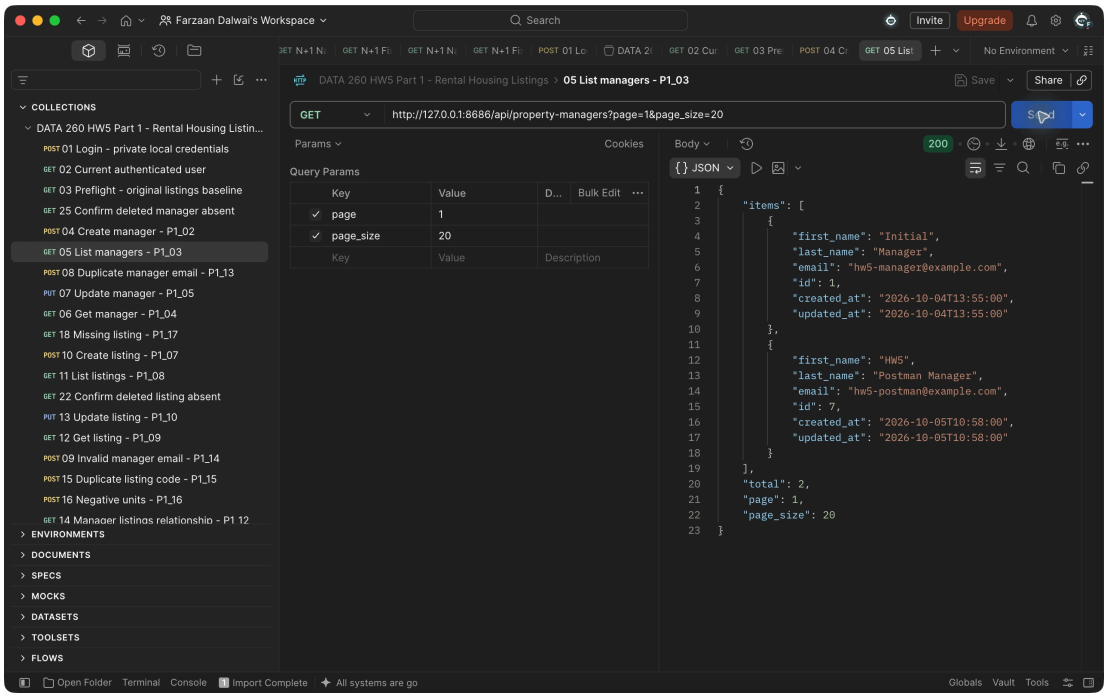
Database constraints



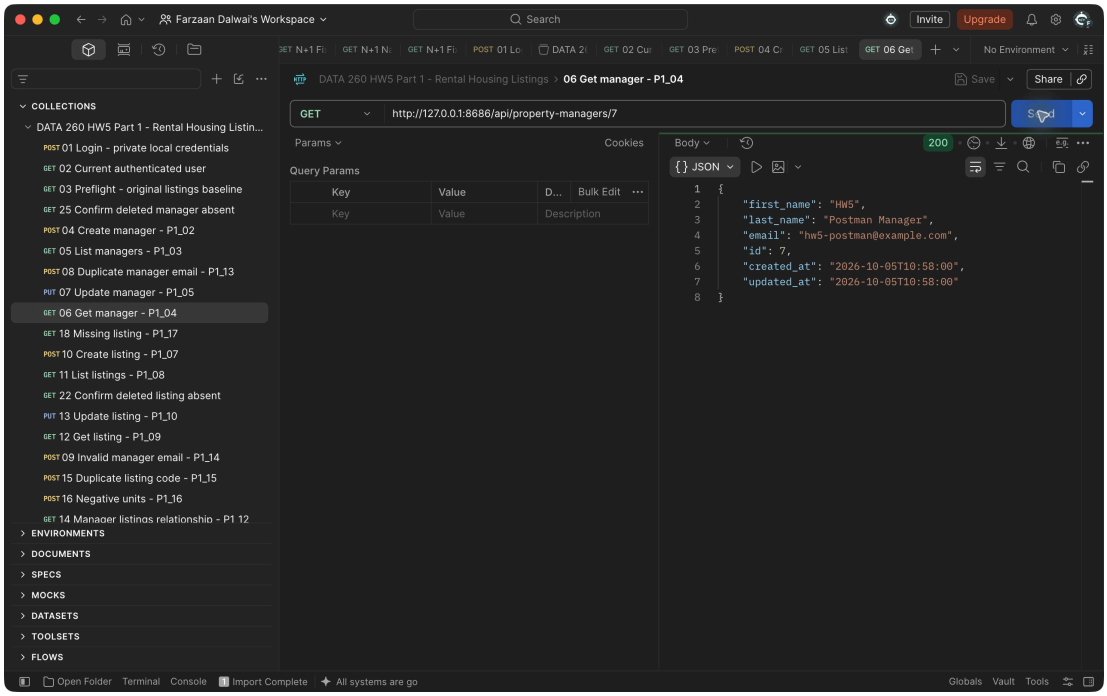
Create manager



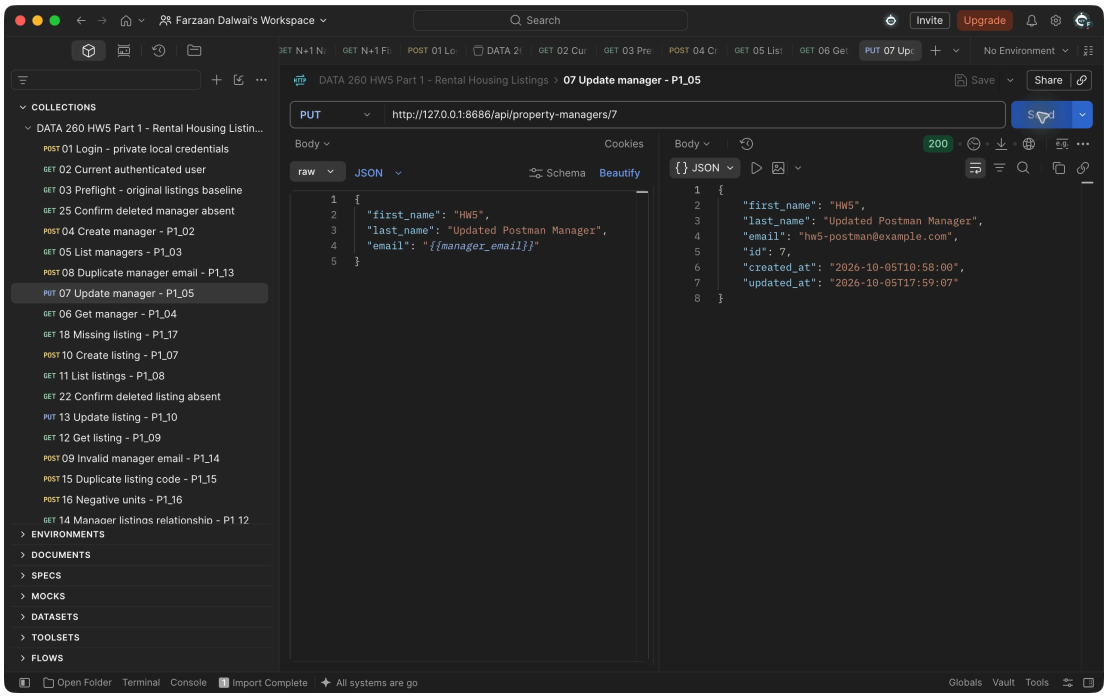
List managers



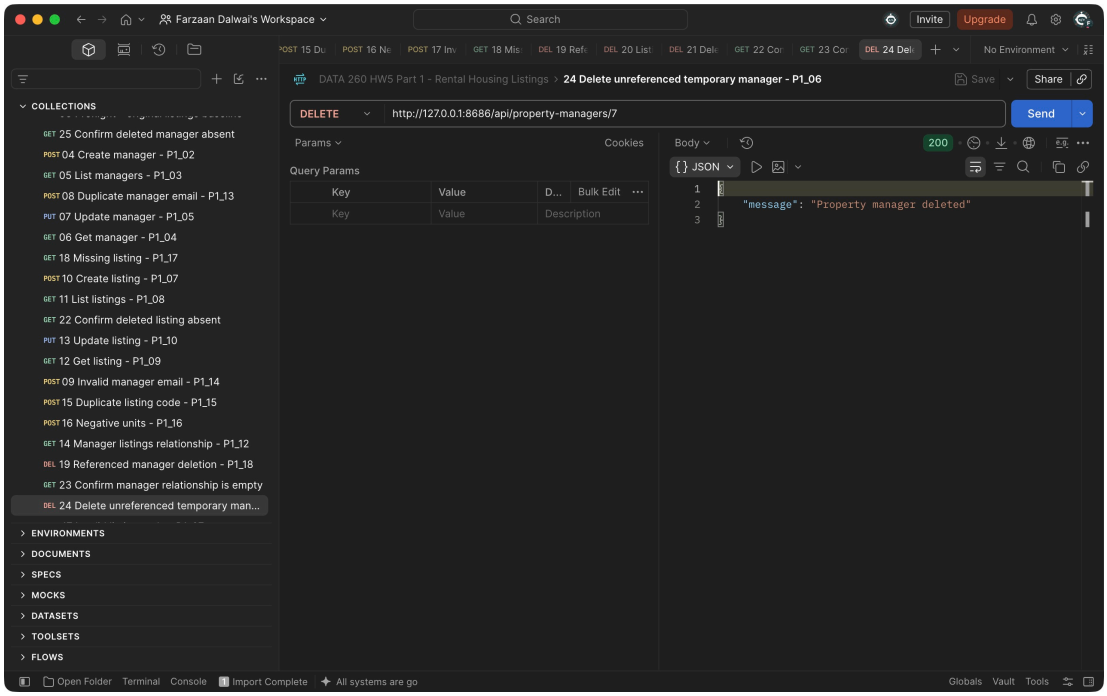
Get manager



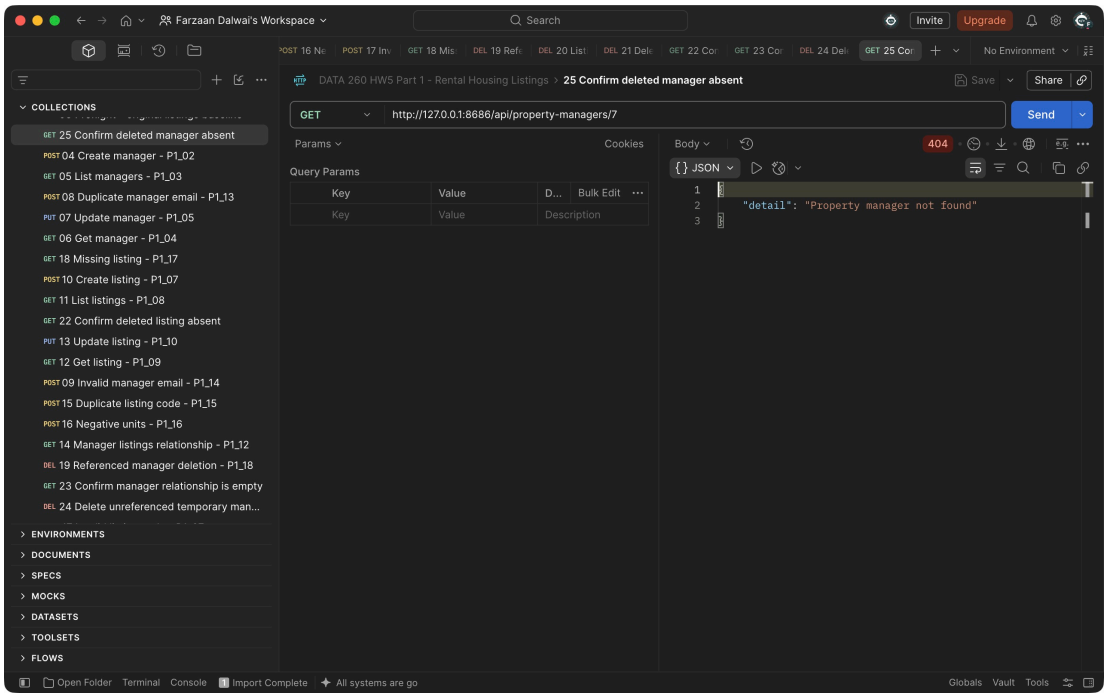
Update manager



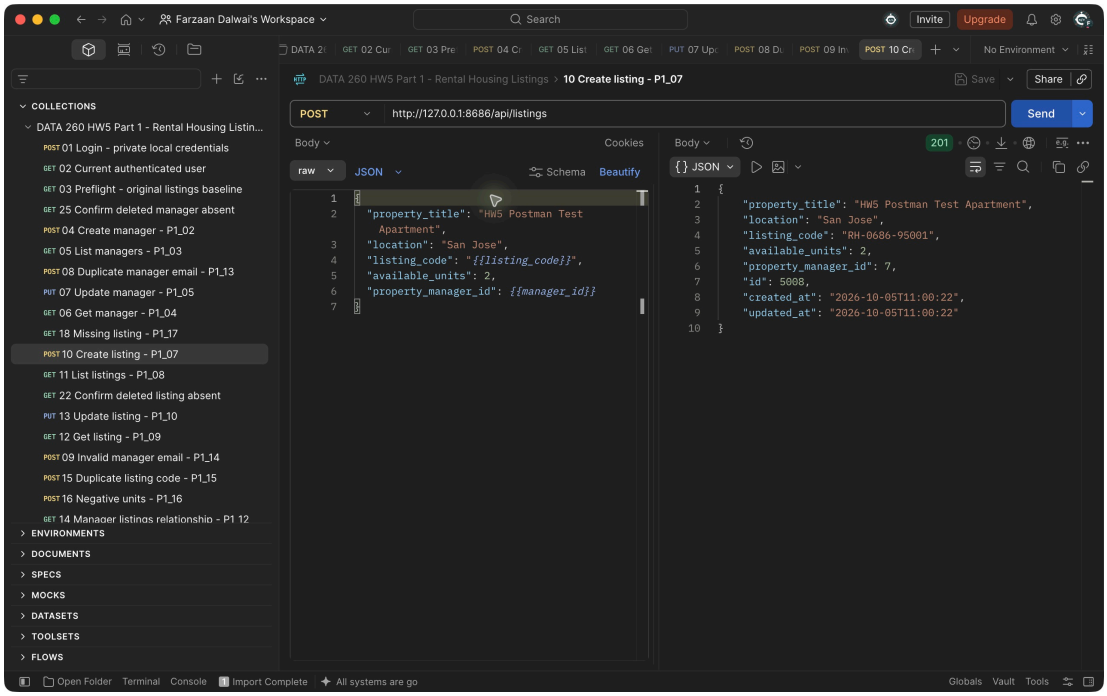
Delete manager



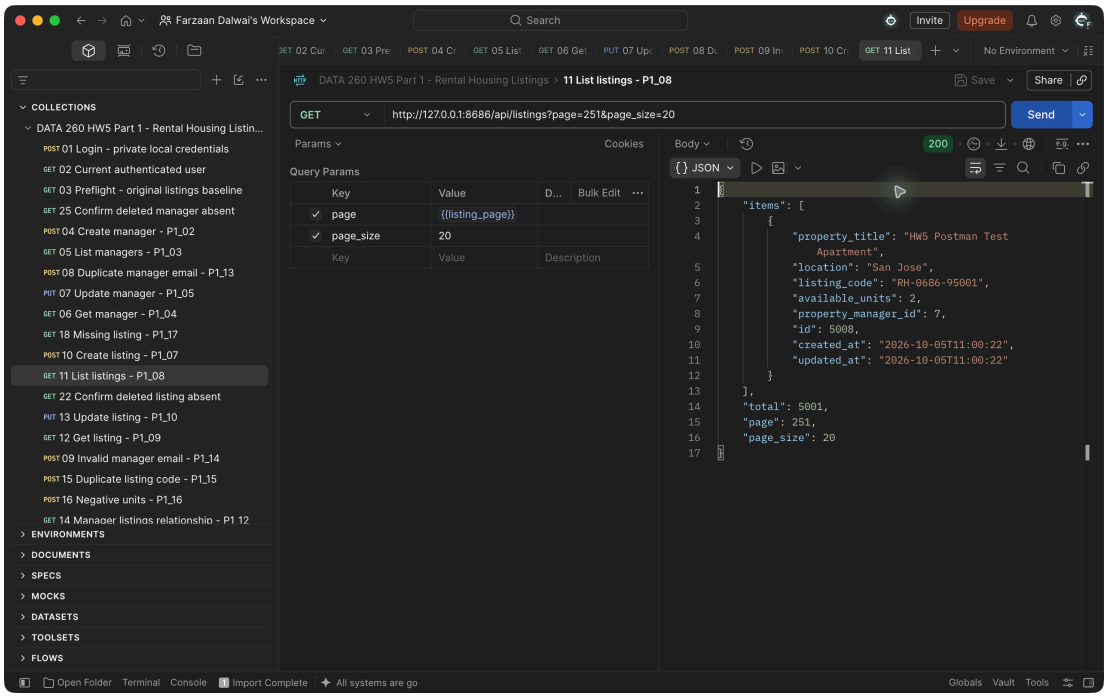
Manager absent



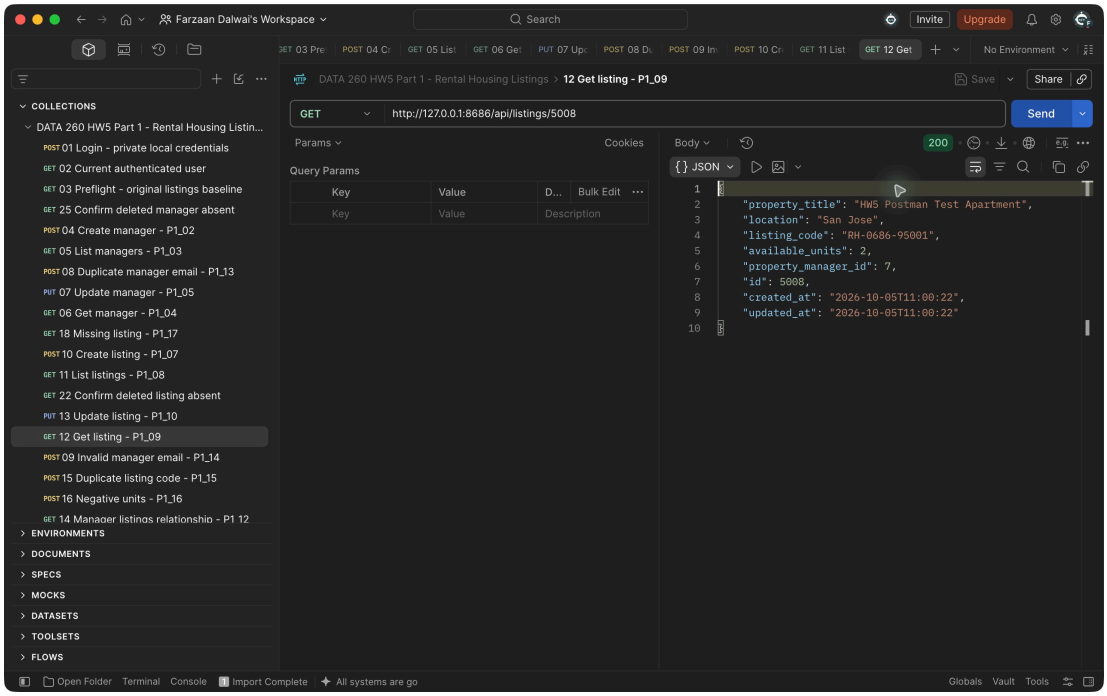
Create listing



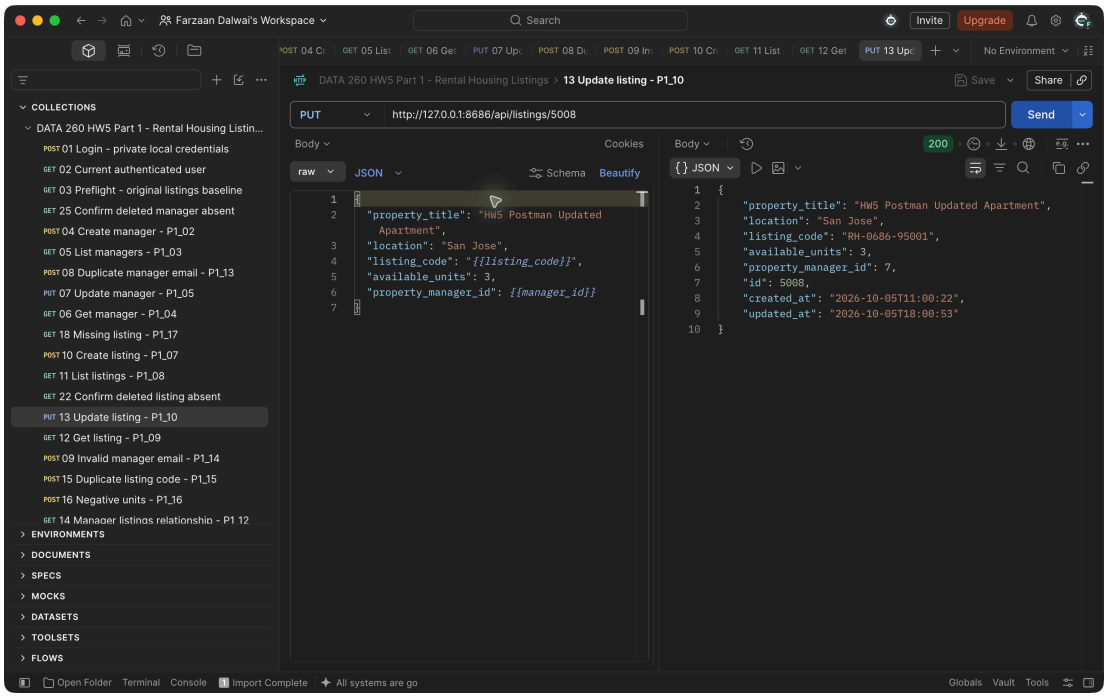
List listings



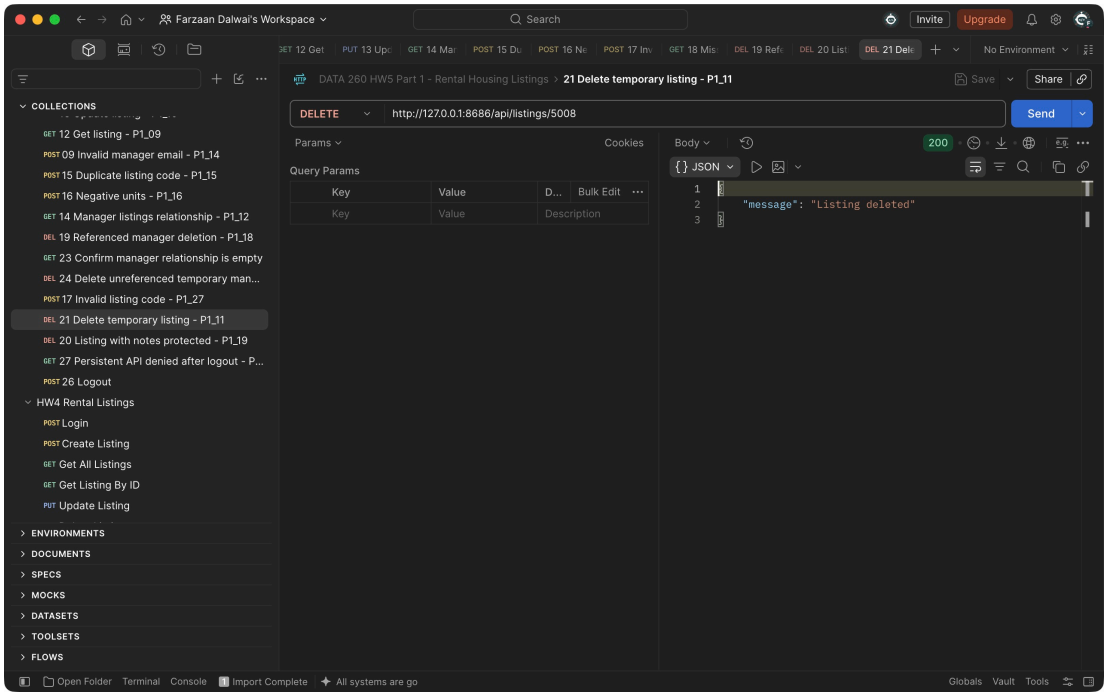
Get listing



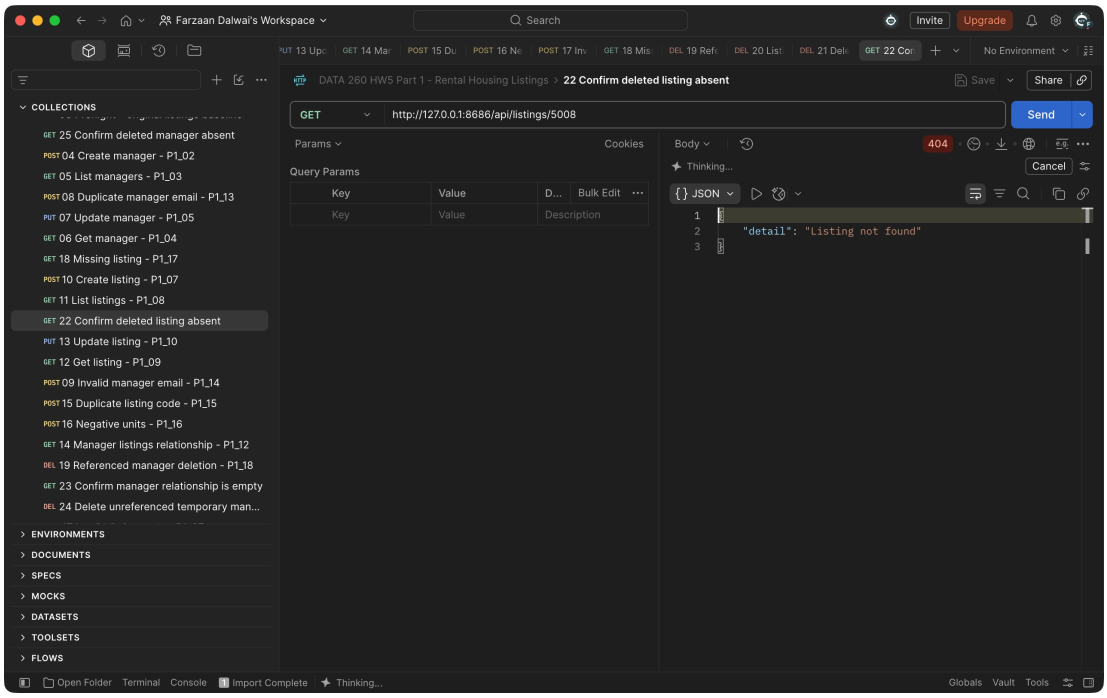
Update listing



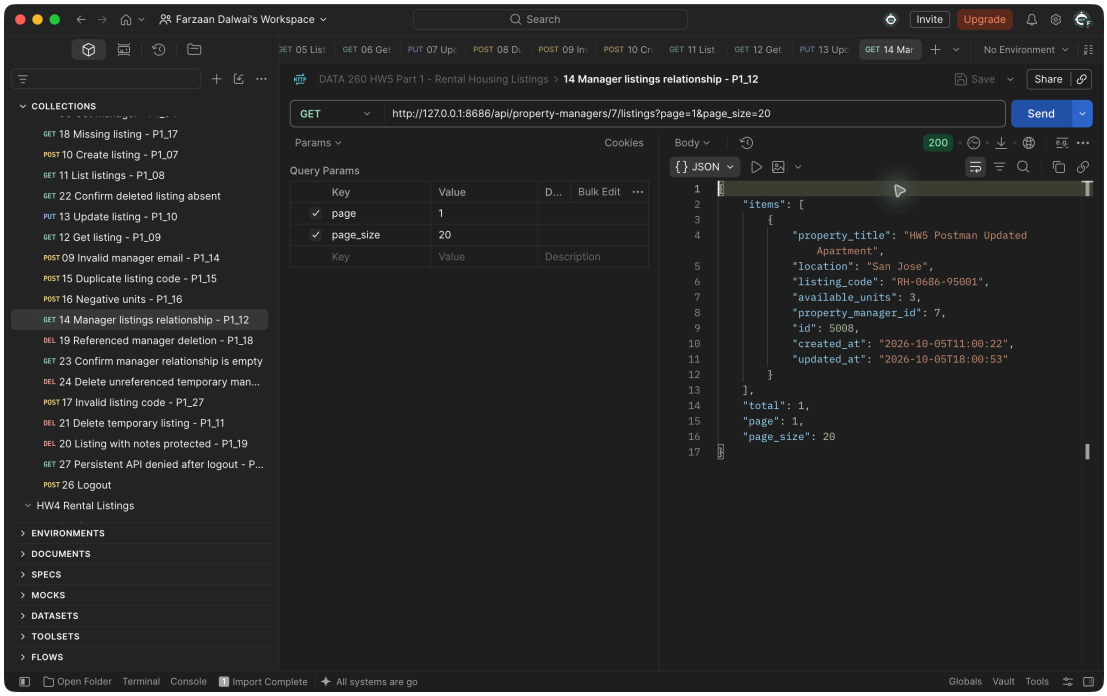
Delete listing



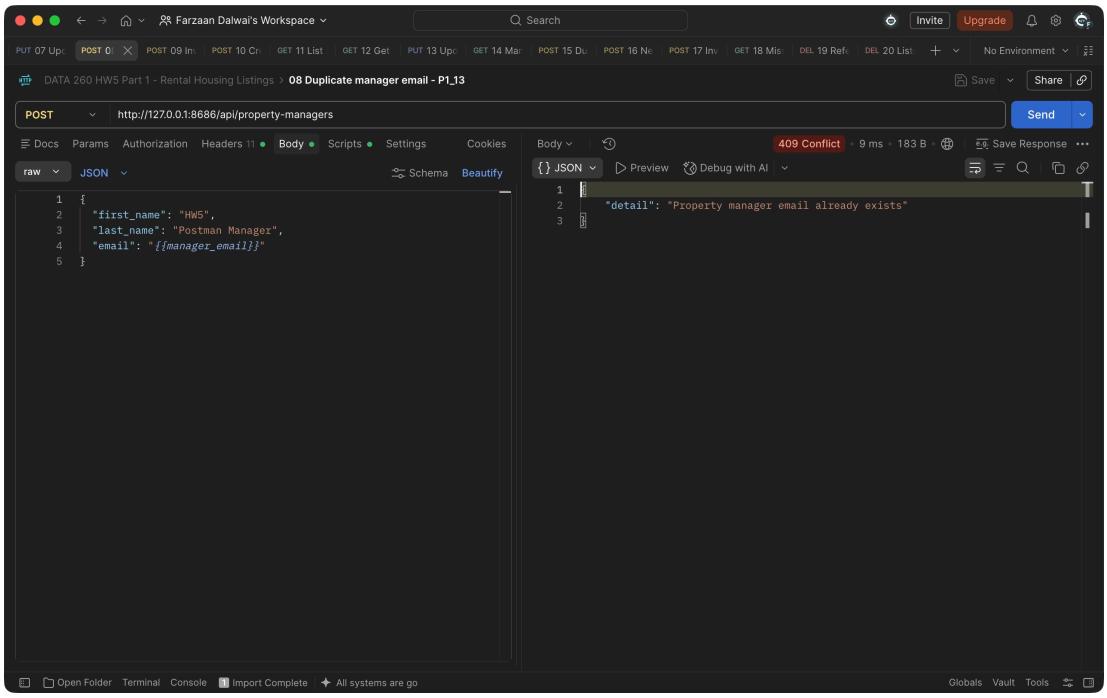
Listing absent



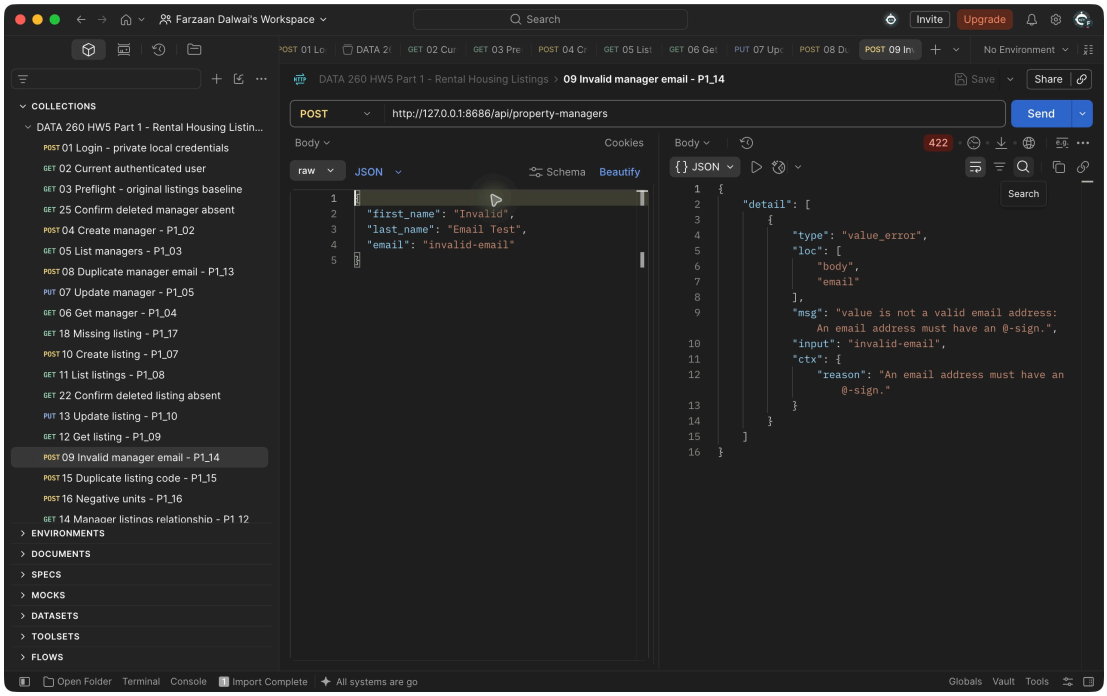
Manager listings



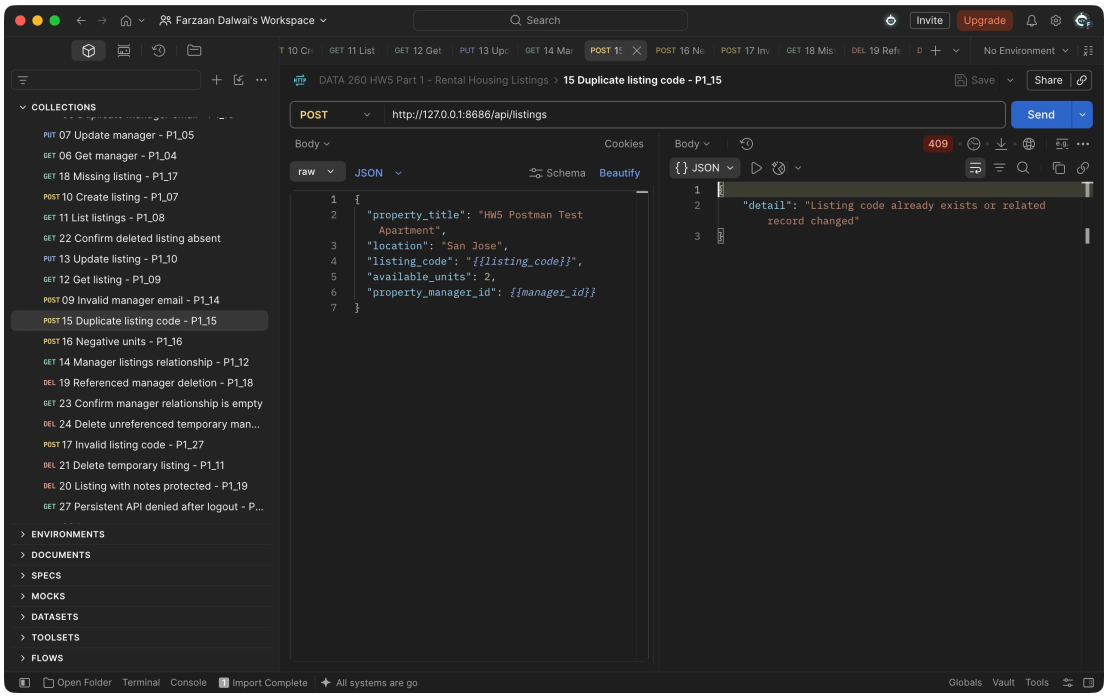
Duplicate email



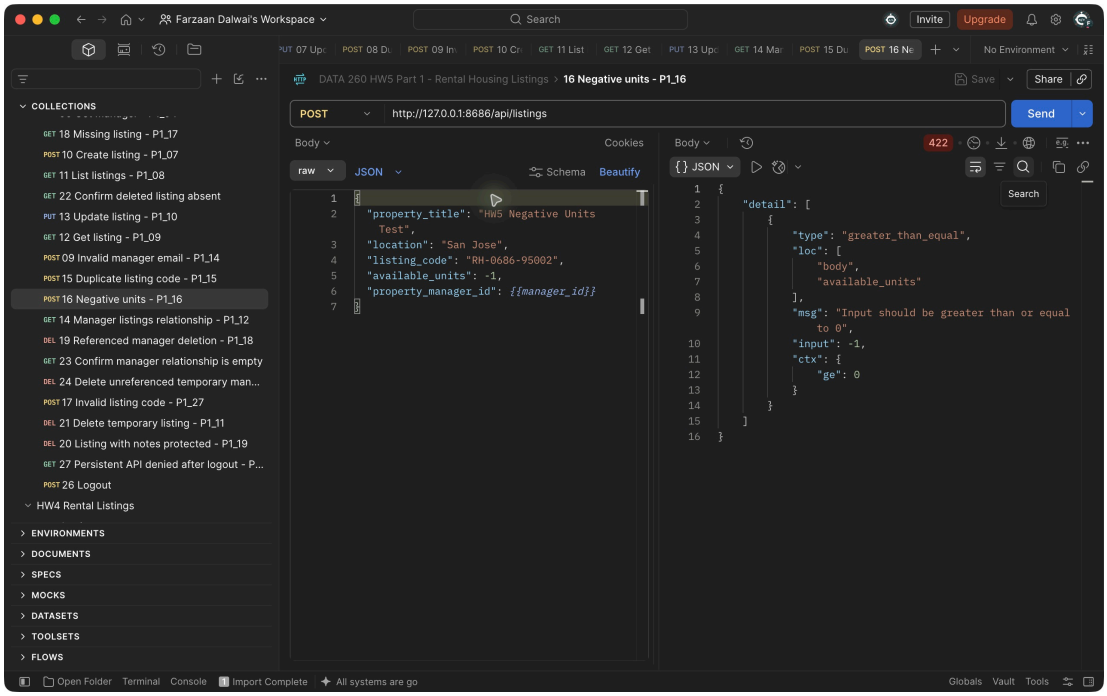
Invalid email



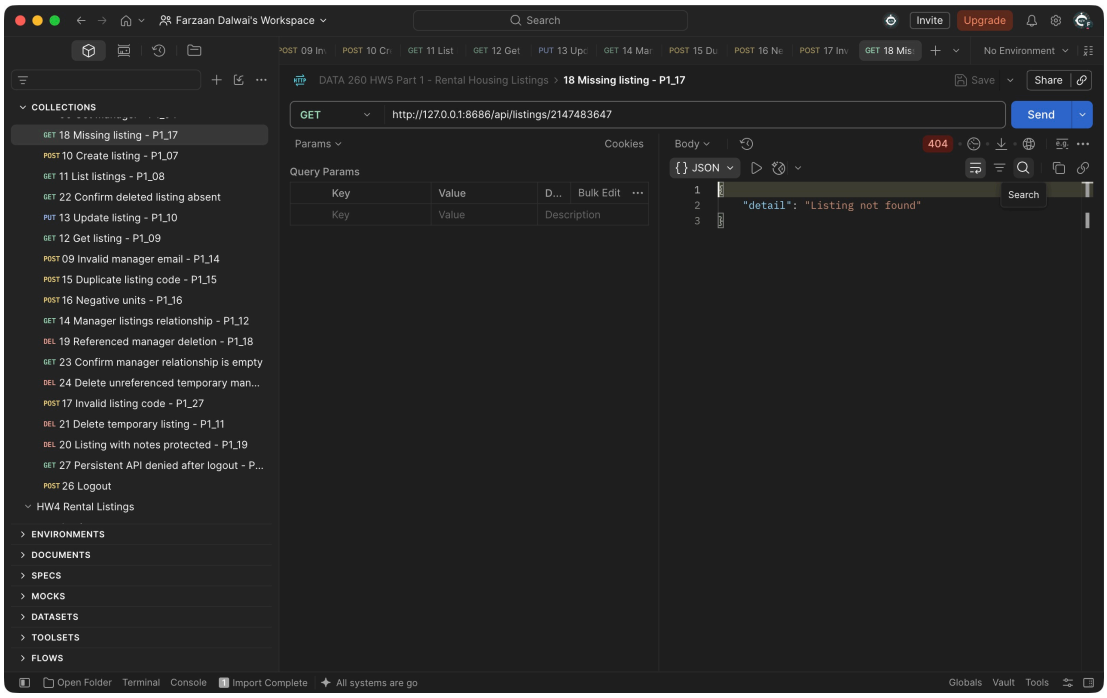
Duplicate listing code



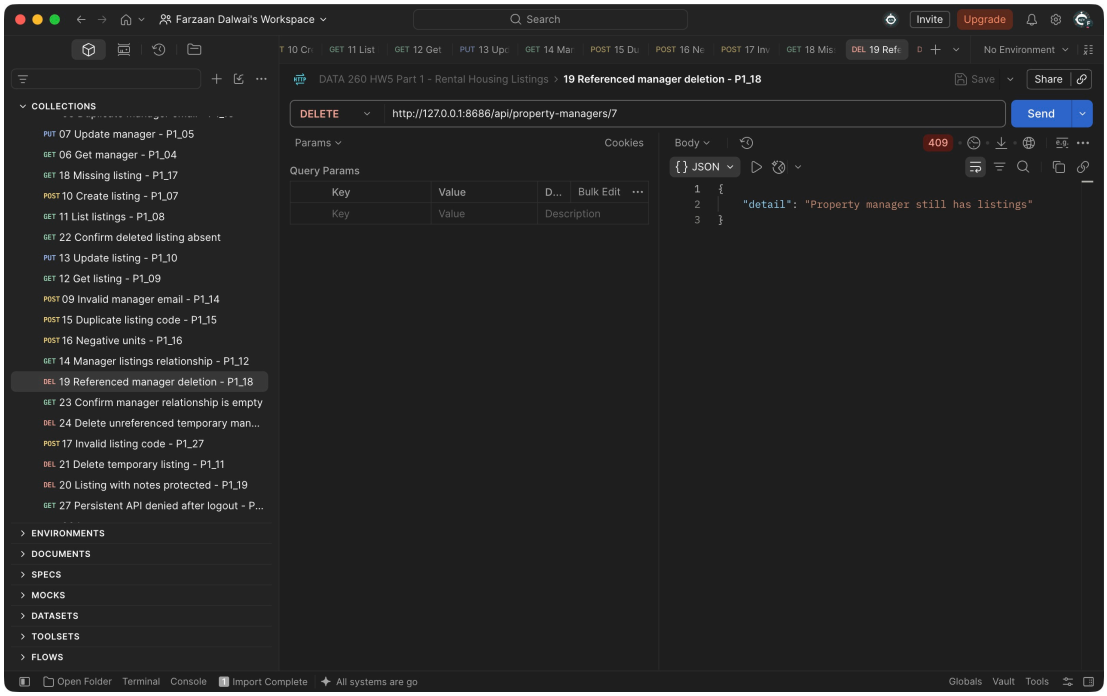
Negative units



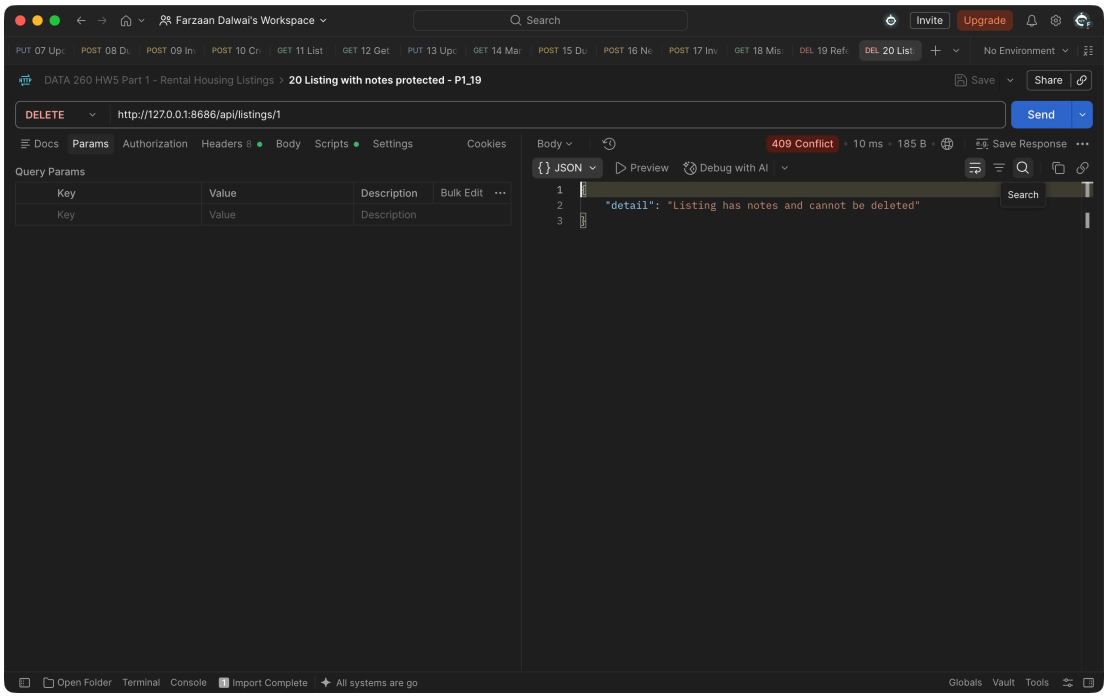
Missing listing



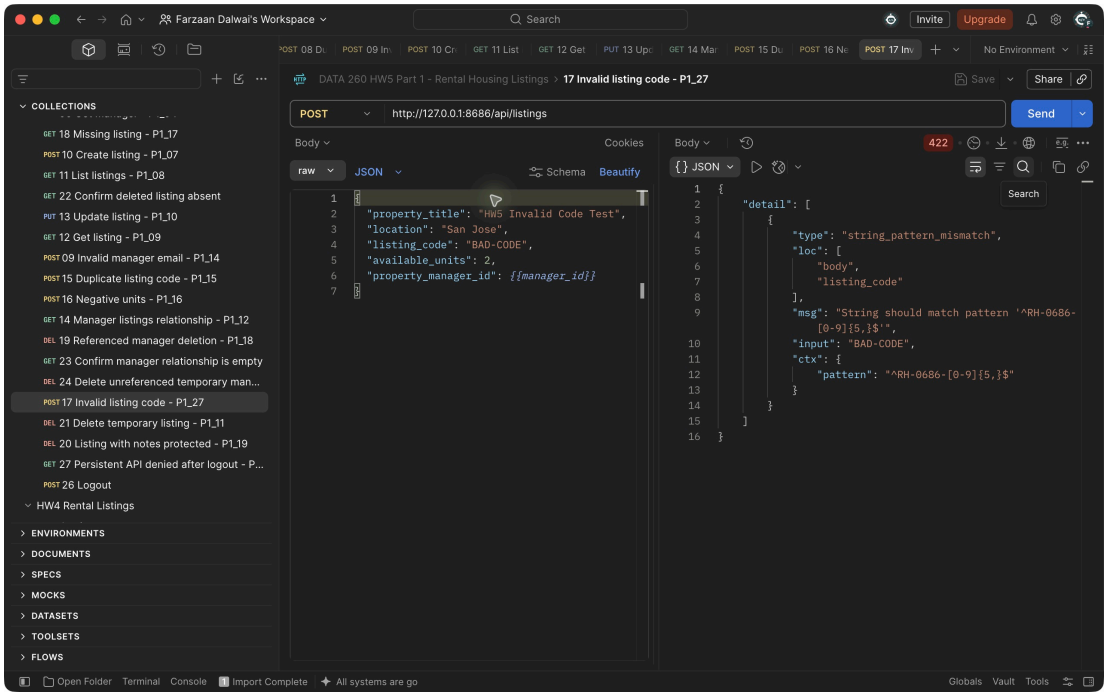
Delete restricted



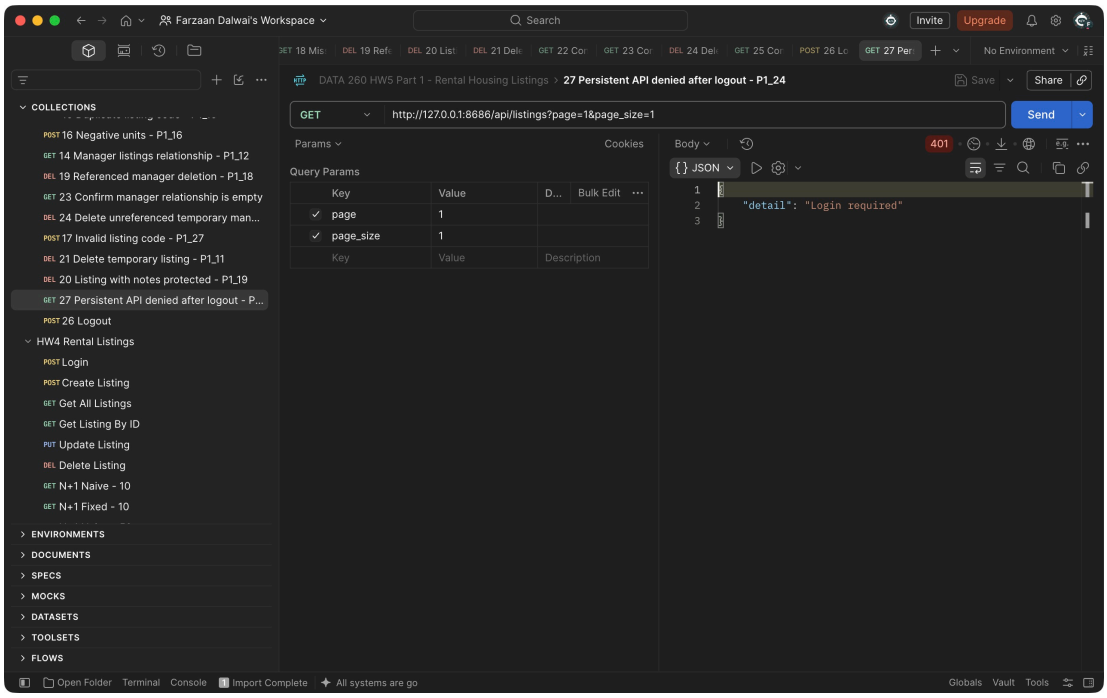
Notes protected



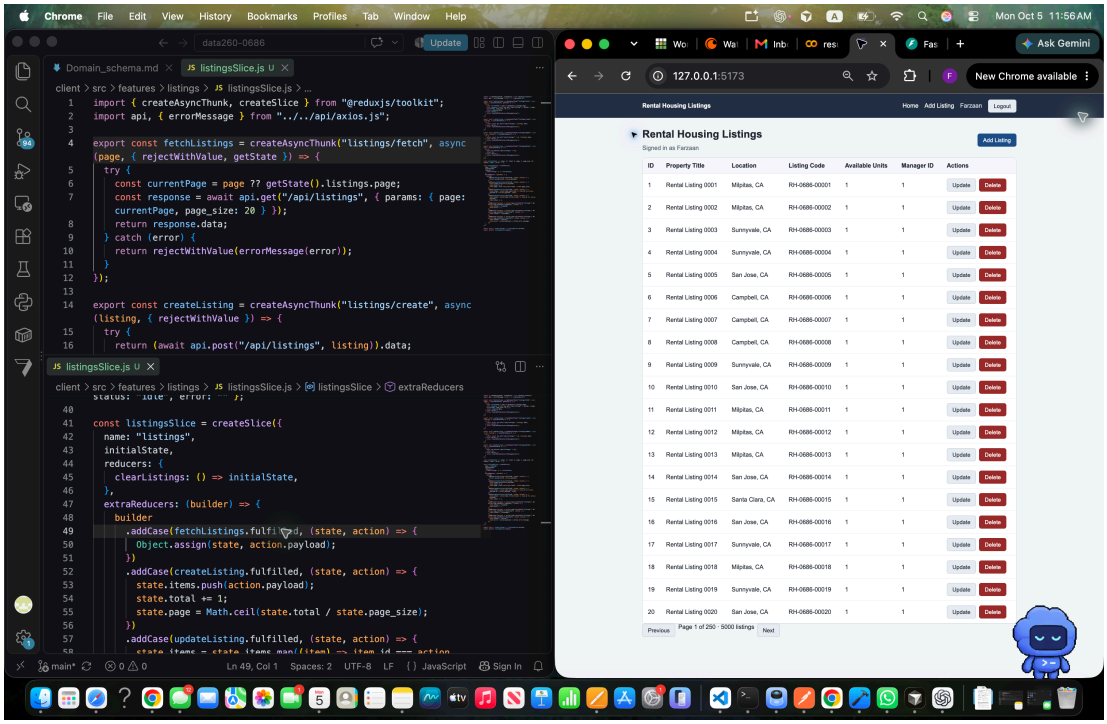
Invalid listing code



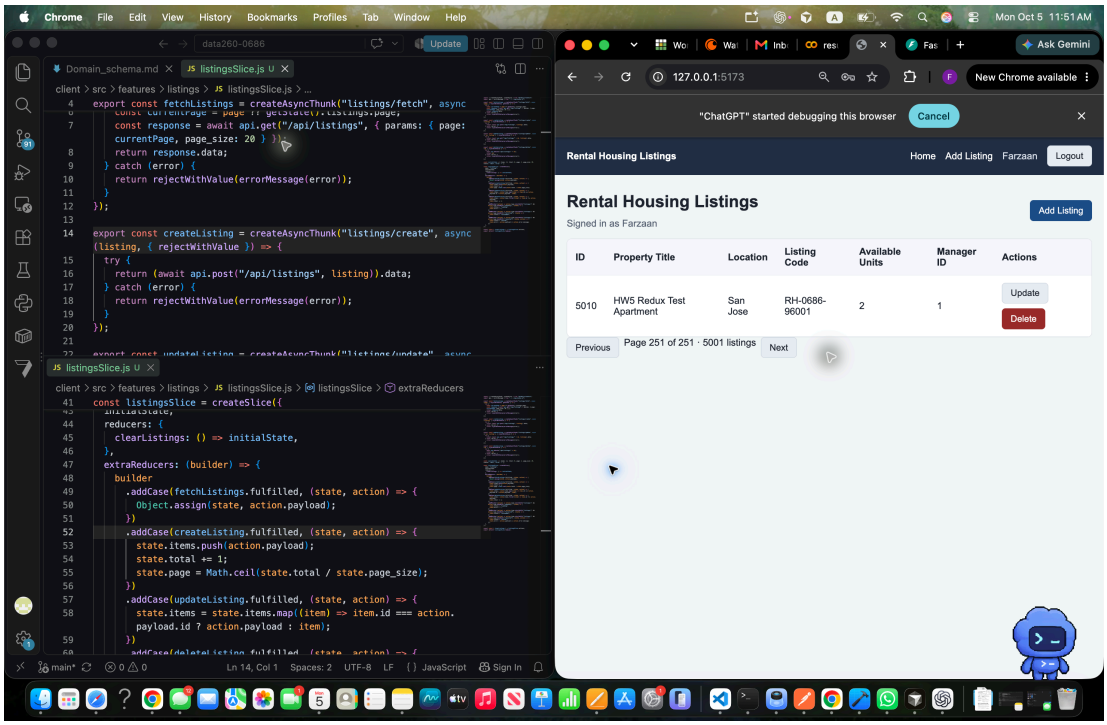
Logged out



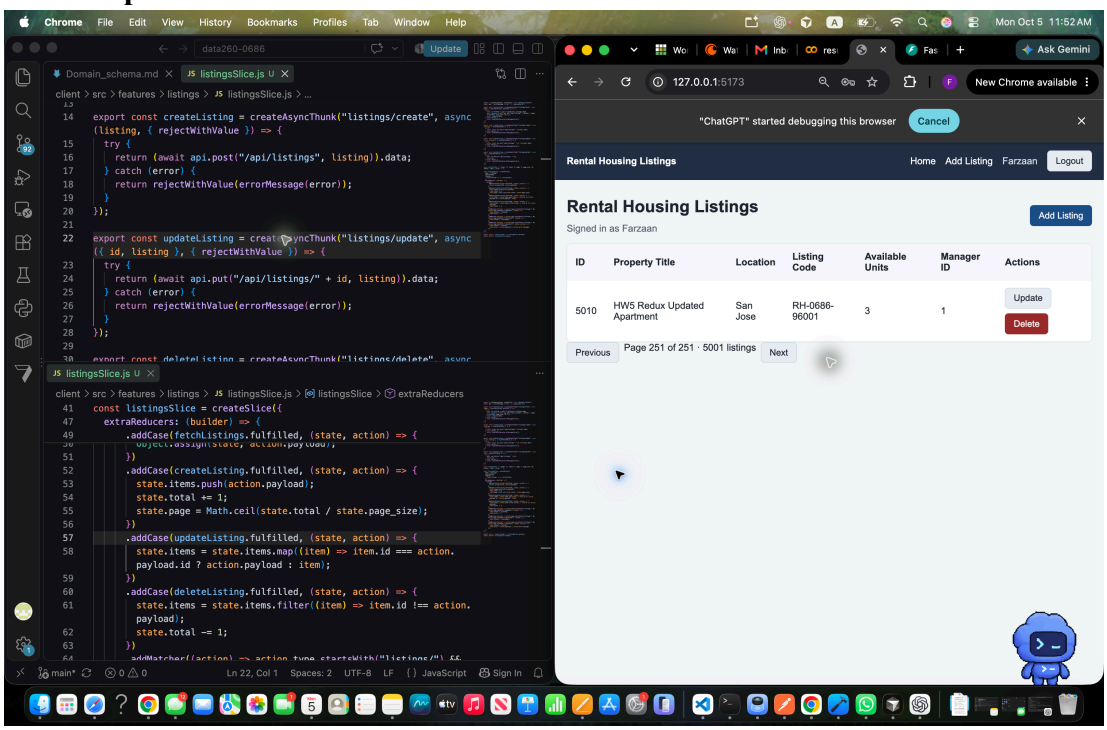
Redux Home



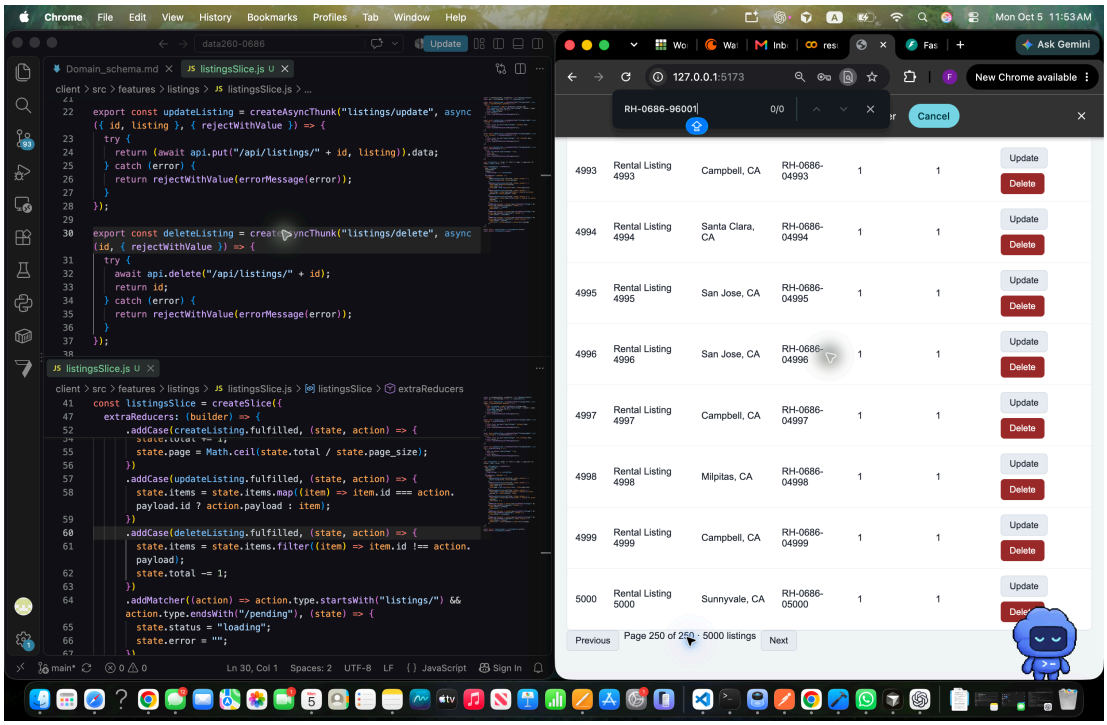
Redux Create



Redux Update

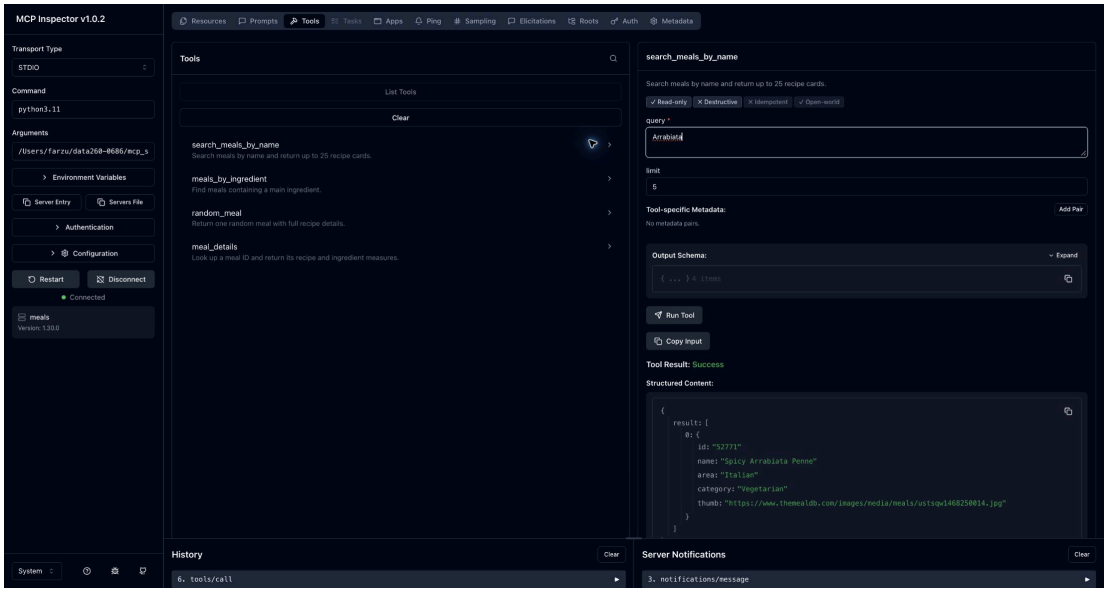


Redux Delete

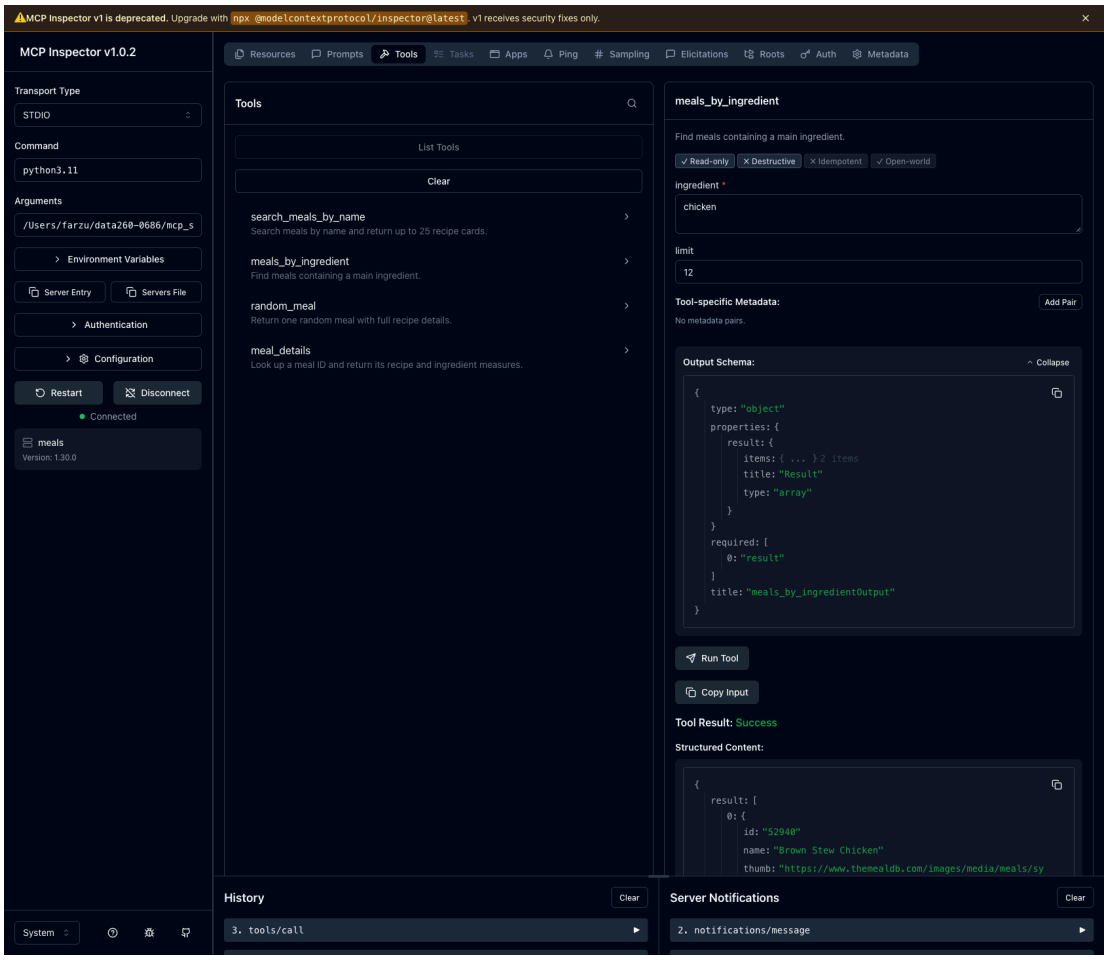


Part 2

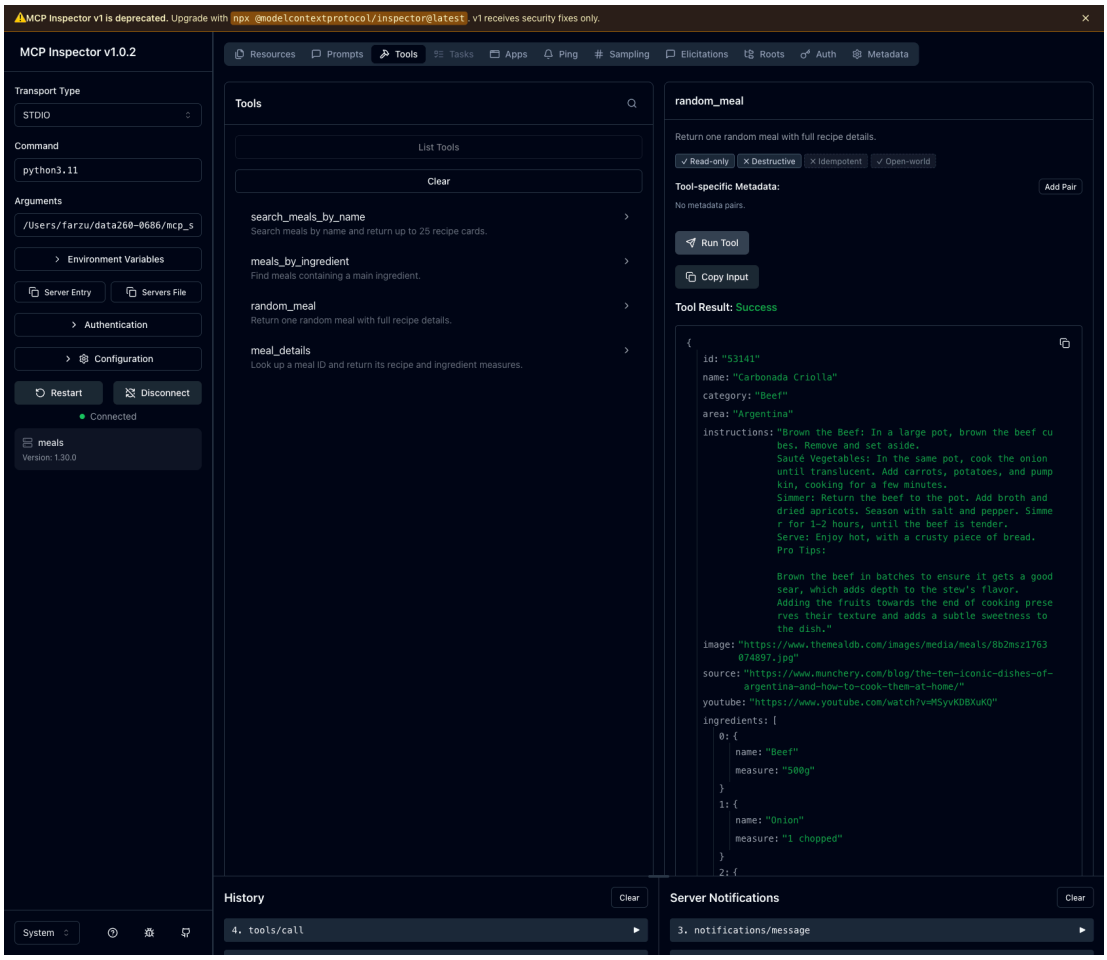
MCP meal search



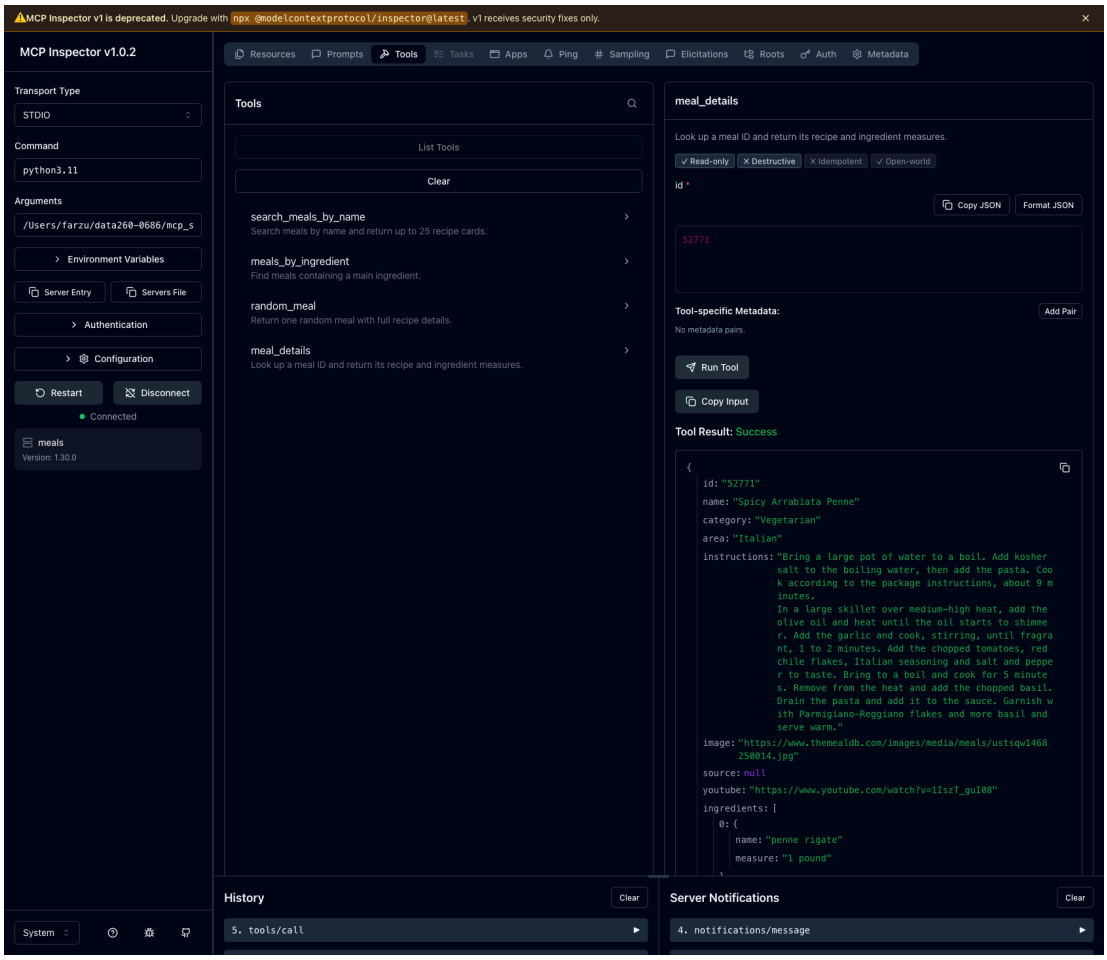
Ingredient search


MCP Inspector v1 is deprecated. Upgrade with npx @modelcontextprotocol/inspector@latest. v1 receives security fixes only.
MCP Inspector v1.0.2
Transport Type
STDIO
Command
python3.11
Arguments
/Users/farzu/data260-0686/mcp_s
Environment Variables
Server Entry
Servers File
Authentication
Configuration
Restart
Disconnect
Connected
meals
Version: 1.30.0
System
Resources
Prompts
Tools
Tasks
Apps
Ping
Sampling
Elicitations
Roots
Auth
Metadata
Tools
List Tools
Clear
search_meals_by_name
Search meals by name and return up to 25 recipe cards.
meals_by_ingredient
Find meals containing a main ingredient.
random_meal
Return one random meal with full recipe details.
meal_details
Look up a meal ID and return its recipe and ingredient measures.
meals_by_ingredient
Find meals containing a main ingredient.
✓ Read-only
✕ Destructive
✕ Idempotent
✓ Open-world
ingredient *
chicken
limit
12
Tool-specific Metadata:
Add Pair
No metadata pairs.
Output Schema:
Collapse
{
type: "object"
properties: {
result: {
items: { ... } 2 items
title: "Result"
type: "array"
}
}
required: [
0: "result"
]
title: "meals_by_ingredientOutput"
}
Run Tool
Copy Input
Tool Result: Success
Structured Content:
{
result: [
0: {
id: "52940"
name: "Brown Stew Chicken"
thumb: "https://www.themealdb.com/images/media/meals/sy
History
Clear
3. tools/call
Server Notifications
Clear
2. notifications/message

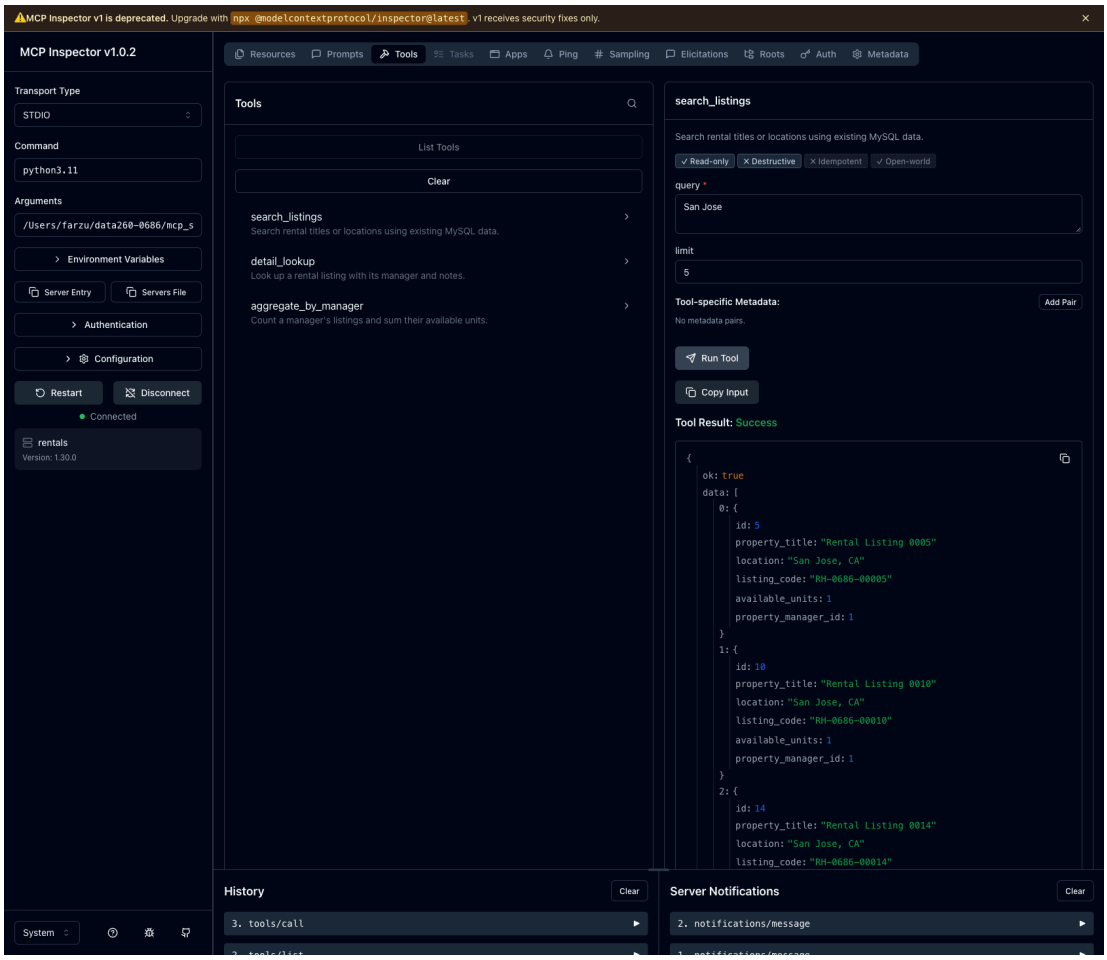
Random meal



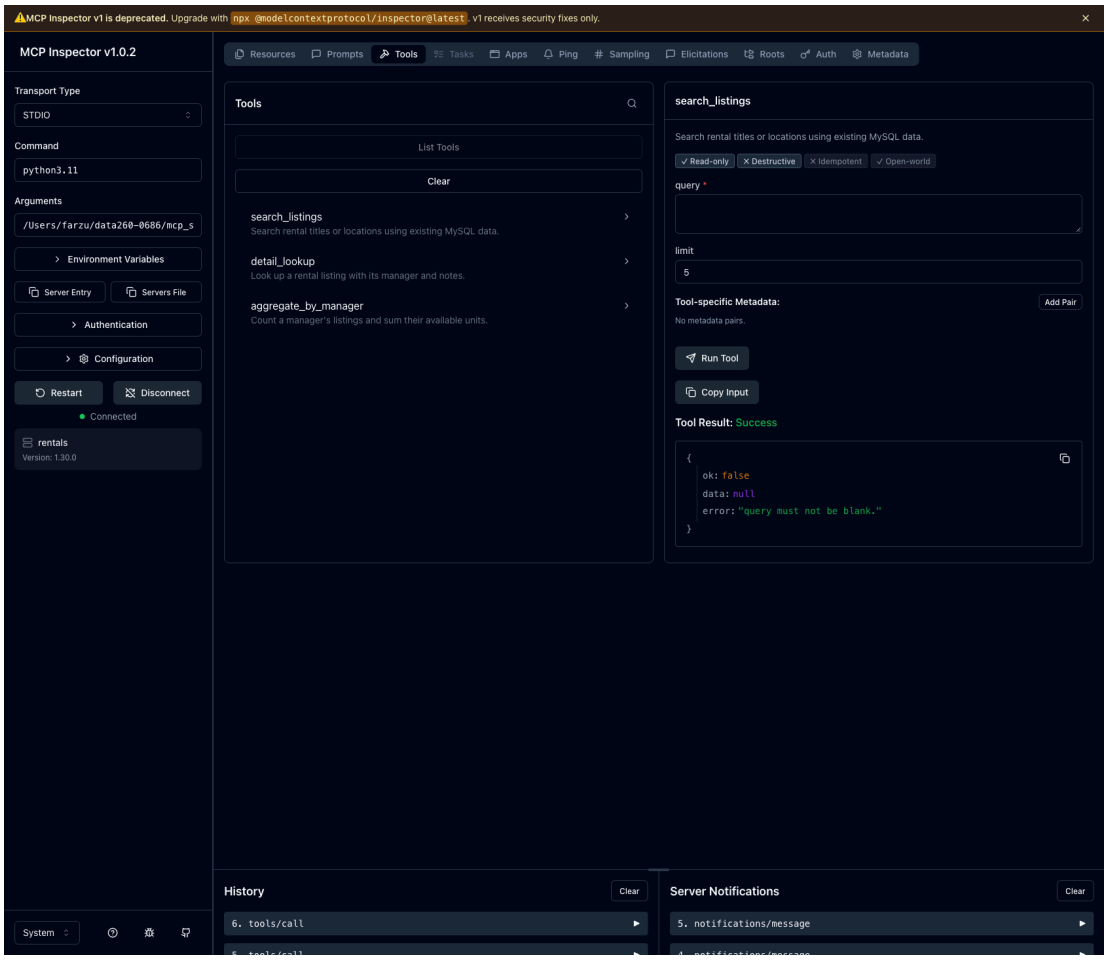
Meal details



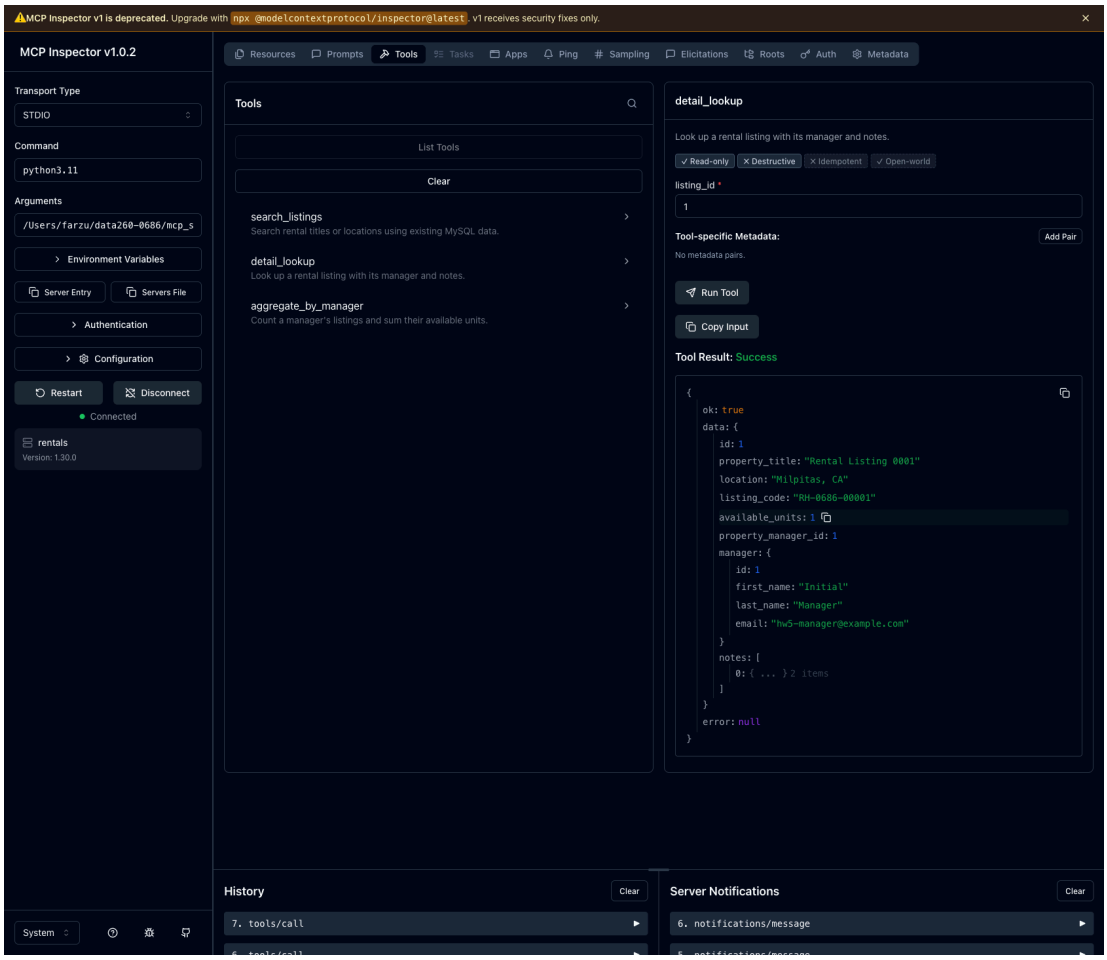
Rental search success



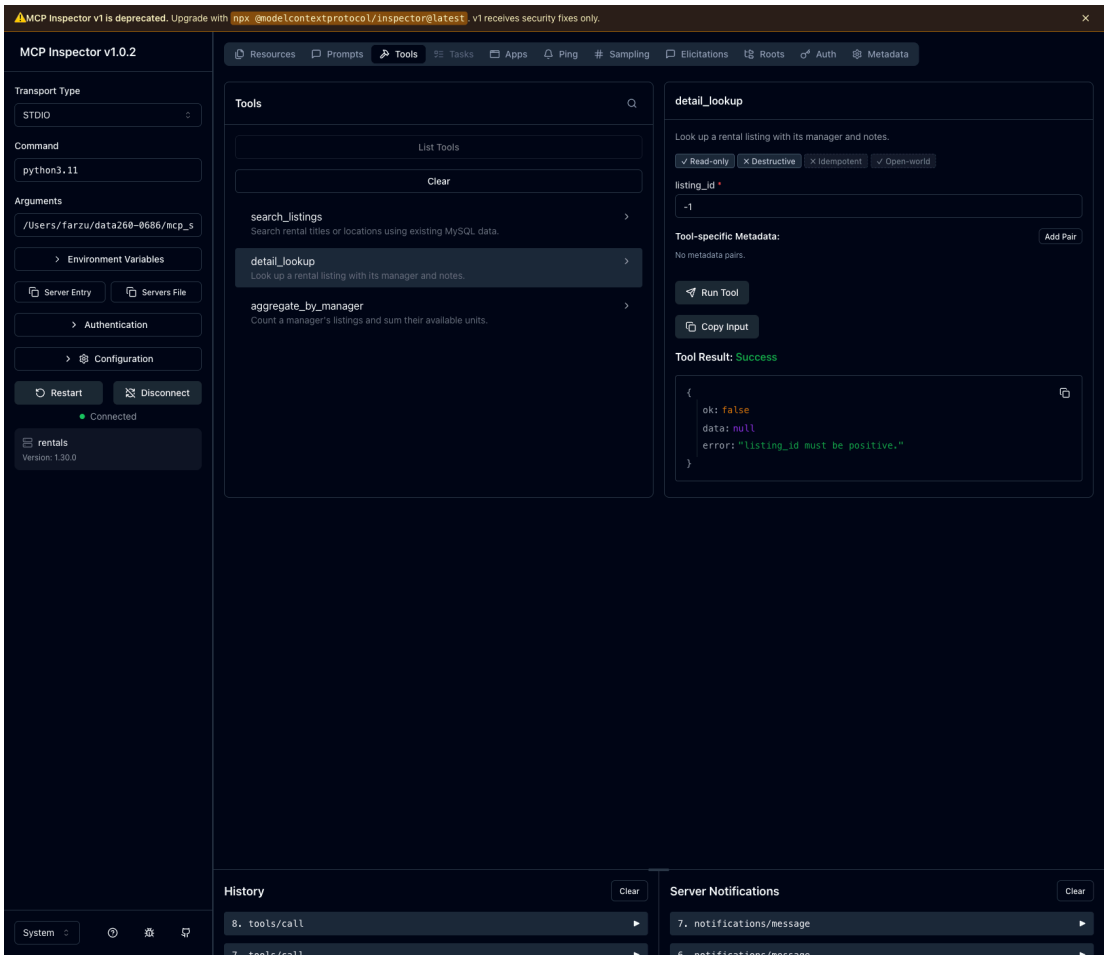
Rental search rejected



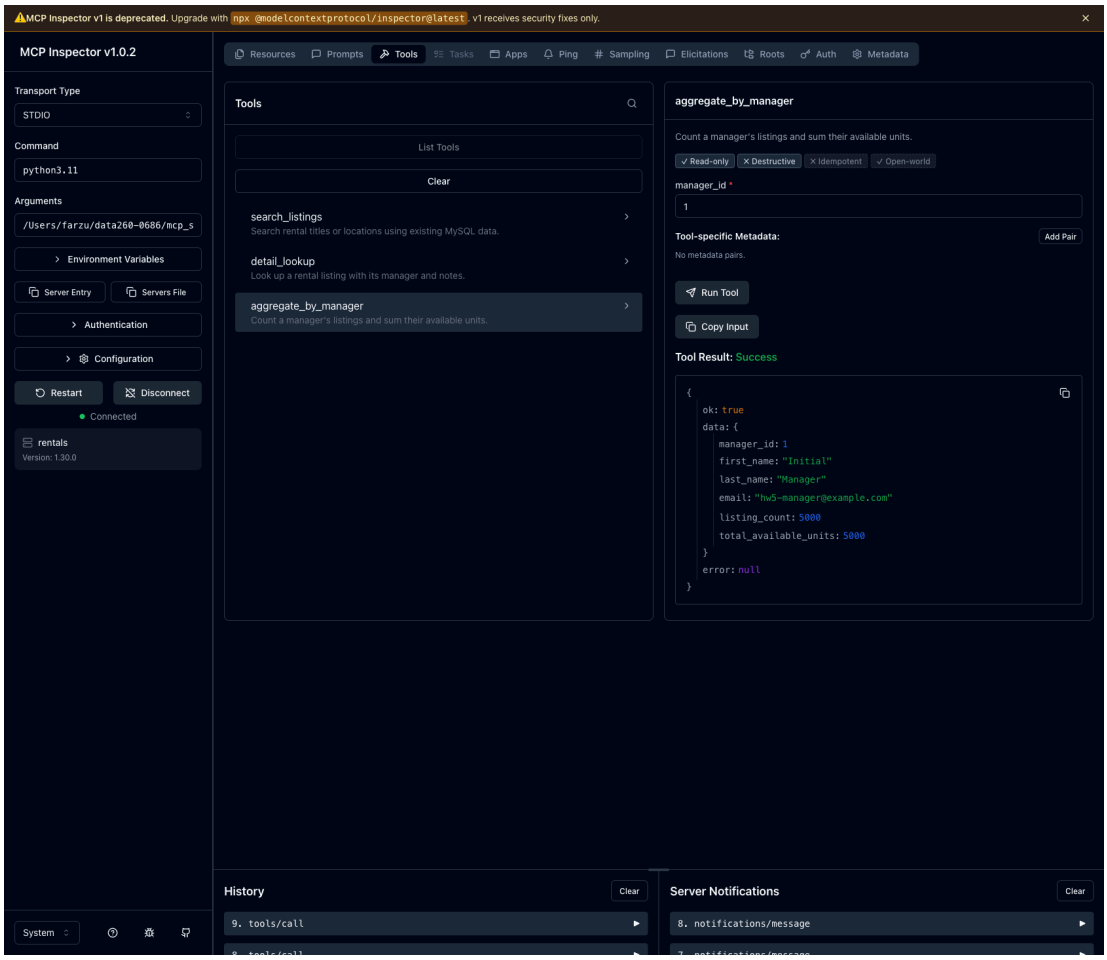
Rental detail



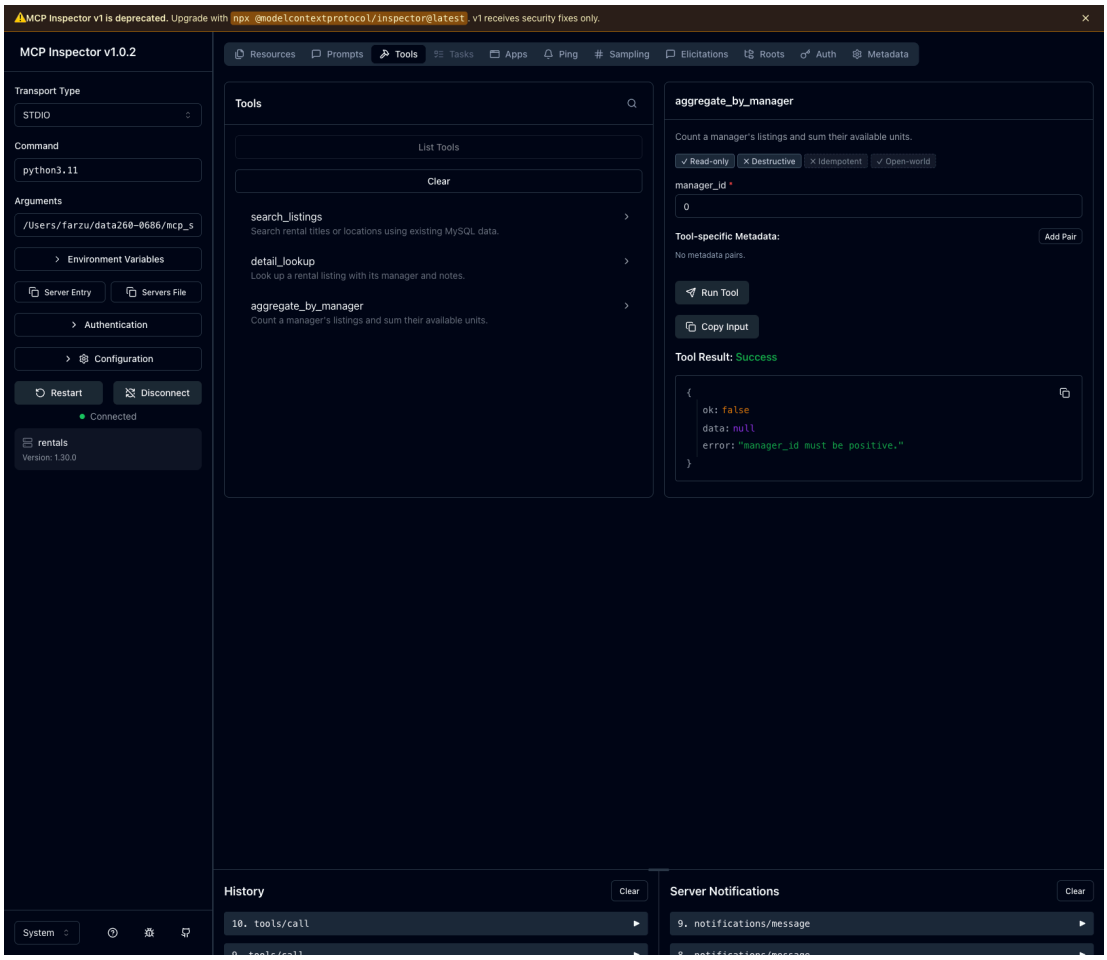
Detail rejected



Manager summary



Summary rejected



Part 3

Each call can try 3 times. The timeout is 0.75 seconds. The wait is 0.02 seconds, then 0.04 seconds, and it stops at 0.08 seconds. The seed is 260686. The numbers below are the final result after retries, not the first try.

Tool	Rejected input	Error
search_listings	query "", limit 5	query must not be blank.
detail_lookup	listing_id -1	listing_id must be positive.
aggregate_by_manager	manager_id 0	manager_id must be positive.

Injected failure rate	Final success rate	Mean latency (ms)	p99 latency (ms)
0%	100% (50/50)	2.447	48.166
20%	100% (50/50)	7.800	27.767
50%	94% (47/50)	19.881	72.002

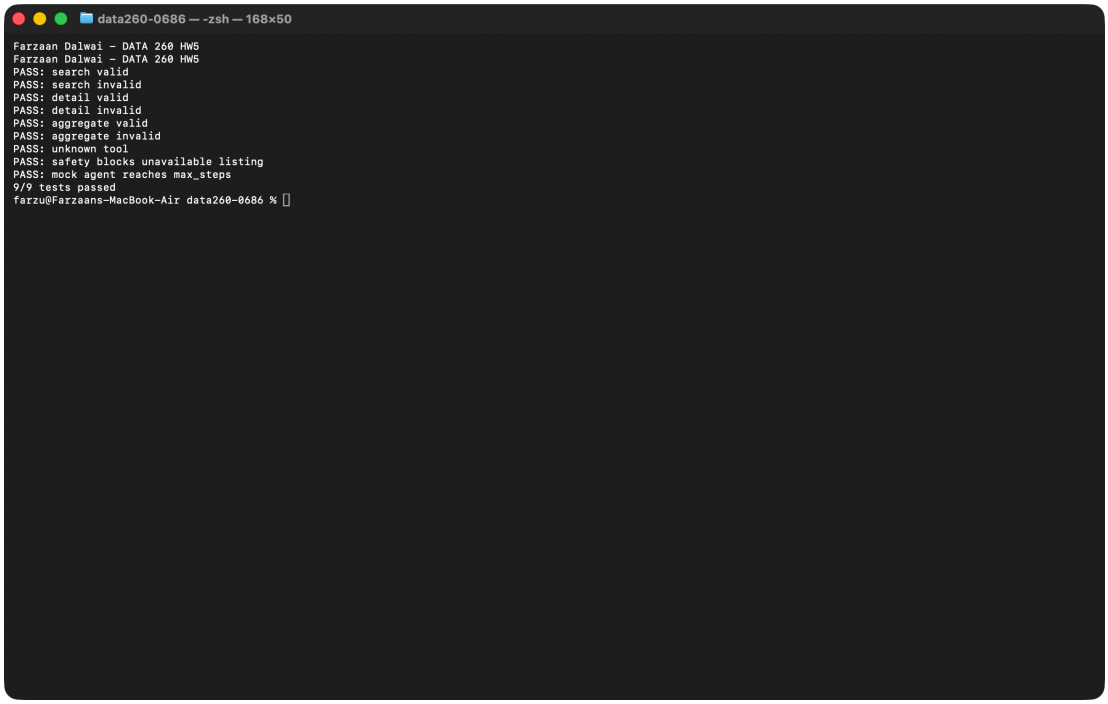
The retry policy worked well at 0% and 20% failure. At 50%, 3 calls still failed after all retries. The delay also increased because more retries were needed. These times stayed under a second, so this is fine when a person is waiting. For batch processing, I would allow more retries and a longer timeout because speed is less important.

Part 4

```
def execute_tool(name, inputs, tools=None):
    selected = tools if tools is not None else default_tools()
```

```
    if name not in selected:
        return json.dumps({"ok": False, "data": None, "error": "Unknown tool."})
    result = selected[name](**normalize_inputs(name, inputs))
    return json.dumps(apply_safety(name, result))
```

Offline tests



9/9 tests passed.

Part 5

Safety rule

```
if units is not None and units <= 0:
    return {"ok": False, "data": None, "error": "Listing is unavailable"}
```

The offline test allowed listing 1. It blocked listing 2 because available_units was 0.

Agent loop

```
if action.get("action") == "final":
    stop_reason = "normal_completion"
    break
name, inputs = tool_request(action)
result = execute_tool(name, inputs)
```

Scenario	Steps	Stop reason	Tool calls
search	2	normal_completion	1
detail	2	normal_completion	1
aggregate	2	normal_completion	1
multi_step	3	normal_completion	2

Reflection

This reflection uses the multi_step record written at 2026-10-05T14:06:24-07:00. The user request was "Find rental listings in San Jose and then give me details for one of them." The harness limit was 4 steps. The run used 3 steps, made 2 tool calls, and stopped with normal_completion.

On step 1 the model returned action "tool", name search_listings, and inputs query "San Jose" with limit 5. execute_tool returned ok true and five listings, ids 5, 10, 14, 16, and 20. Their locations were San Jose, CA, and each available_units value was 1. The harness continued because the request also asked for details and the model had not returned a final answer.

On step 2 the model selected detail_lookup with listing_id 5, the first id from the search. The tool returned ok true. The listing is Rental Listing 0005 in San Jose, CA, code RH-0686-00005, with available_units 1 and property_manager_id 1. The manager is id 1, first name Initial, last name Manager. The notes list has one note, id 5, with the text "Related note 005". available_units was 1, so the unavailable-listing safety rule did not apply.

On step 3 the model returned action "final". The answer said the listing is in San Jose, CA, the property manager is Initial Manager, and there is one available unit. Those statements match the detail result. The answer did not add a street address. The harness treated the final action as a normal completion and did not take another step.

AI Use

I mainly used AI for code debugging. I also used it to understand some concepts step by step and to clarify which screenshots and evidence the homework required. I personally ran the code, tests, experiments, database checks, and MCP calls, and I captured and checked the screenshots.

One generated agent response included an unsupported unit count and a street address. I found that by comparing the response with the logged execute_tool result. I tightened the grounding instruction, reran the scenarios, and checked them again.